

Phân tích lỗ hổng của script do AI sinh ra

HW04 §6 yêu cầu: báo cáo những gì AI làm sai hoặc bỏ sót trong quá trình chuyển 30 case thủ công sang từ HW02 cộng 10 case thiết kế mới trong HW04 thành script Playwright, và giải thích **vì sao** AI bỏ sót: chất lượng prompt, giới hạn của mô hình, hay đặc điểm riêng của tính năng mà AI không quan sát được. Mỗi mục dưới đây gắn với một entry có số thứ tự trong `./reports/ai-audit-report.md`, lấy đúng từ những gì đã xảy ra trong Session 1–4, không phải danh sách lỗi AI thường gặp nói chung.

1. Nhóm lỗi lặp lại nhiều nhất: giả định giao diện trước khi đọc DOM thật

Đây là nhóm đông nhất và nguy hiểm nhất, xảy ra ở cả ba feature: mọi bản nháp locator do AI viết trước khi recon trên build đang chạy đều sai ở ít nhất một chỗ, và không lỗi nào trong nhóm này lộ ra bằng cách đọc code; tất cả chỉ lộ khi chạy thử.

1.1. FR-02 — Entry #8

AI sinh ra: `getByLabel(/email/i)` cho ô nhập email, và giới hạn khối thông báo lỗi trong phạm vi `<form>`.

Vì sao sai: Form đăng nhập không gắn `<label for>` với input, input cũng không có `name / id / placeholder`, nên `getByLabel` không khớp gì cả trên một DOM như vậy. Banner lỗi render bên ngoài `<form>`, nên locator giới hạn trong form sẽ luôn "không tìm thấy phần tử", khiến người đọc dễ kết luận nhầm rằng SUT không hiển thị lỗi.

Đã sửa: Neo theo chữ hiển thị cạnh ô nhập thay vì `getByLabel`; đưa locator lỗi ra ngoài phạm vi `<form>`. Ghi chú thẳng trong `login.page.ts` lý do không dùng `getByLabel`, để lần bảo trì sau không "dọn dẹp" ngược lại.

Vì sao AI bỏ sót: Đặc điểm riêng của tính năng: không có cách nào suy ra cấu trúc DOM thực tế (label không liên kết, banner nằm ngoài form) từ đặc tả hay từ tên biến/route. Đây thuộc nhóm "giới hạn quan sát", không phải giới hạn suy luận của mô hình.

1.2. FR-10 — Entry #11, #12

AI sinh ra (theo bản nháp trong kế hoạch 05/08): Schema dùng `action: 'set-status' + targetStatus`, page object dùng `statusSelect(orderId) / selectOption(status)`, và điều hướng thẳng bằng URL (`:5174/orders`, `:5173/orders`).

Vì sao sai: Ba giả định đều sai với build thật, và mỗi giả định lộ ra theo cách khác nhau: - Không có `<select>` — admin dùng một nút riêng cho mỗi chuyển đổi hợp lệ. Nếu giữ nguyên schema, các case "chuyển đổi không hợp lệ" không có chỗ để biểu diễn kỳ vọng. - App admin không route theo URL: mở `:5174/orders` vẫn render Dashboard. `page.goto()` "thành công" (không lỗi 404) nên sai sót này không

làm test crash ngay, mà làm mọi bước sau đó thao tác nhằm màn hình. - Trang lịch sử đơn của User nằm ở `/profile`, không phải `/orders`. - Hai origin dùng hai khoá lưu token khác nhau (`token` so với `adminToken`). Seed nhằm khoá khiến app luôn ở màn hình đăng nhập mà không báo lỗi rõ ràng — một dạng lỗi im lặng, khó phát hiện nhất trong toàn bộ nhóm này.

Đã sửa: Đổi từ vưng `assertion` sang các khái niệm quan sát được (`expectedControls`, `controls-exactly`); `AdminOrdersPage.goto()` điều hướng bằng cách bấm mục Đơn hàng ở sidebar; `MyOrdersPage.goto()` trở `/profile`; fixture seed đúng khoá token theo từng origin. Locator dòng đơn đổi từ `filter({ hasText: String(orderId) })` sang regex neo chính xác `#<id>`; bản nháp cũ sẽ khớp nhầm `#3` với cả `#30` và với chuỗi `3` xuất hiện trong số tiền hoặc ngày tháng.

Vì sao AI bỏ sót: Bản nháp trong kế hoạch được viết trước khi có quyền truy cập vào build đang chạy (giai đoạn lập kế hoạch, 05/08), tức là một giới hạn về *thời điểm quan sát* chứ không phải giới hạn suy luận: AI thiết kế theo mẫu UI phổ biến nhất (dropdown trạng thái, route theo URL) vì không có dữ liệu nào khác để dựa vào tại thời điểm đó.

1.3. FR-13 — Entry #17

AI sinh ra: Nhánh `access-denied-non-admin` gọi `seedUserToken(page, user.token)`. Hàm này bơm token vào khoá `token` ở origin shop (`:5173`) rồi điều hướng về chính origin đó.

Vì sao sai: Case cần kiểm tra là "app admin có tự chặn token không phải admin hay không", nhưng `seedUserToken` không hề chạm tới app admin — `dashboard.dashboardHeading` được kiểm tra trên trang chủ shop, nơi chắc chắn không có heading đó. Case pass, nhưng pass sai lý do: nó không hề kiểm tra thứ nó tuyên bố kiểm tra. Đây là dạng lỗi nguy hiểm nhất trong ba lần recon-sai của toàn bài: không phải test đỏ oan, mà là **test xanh giả**, chỉ lộ ra khi so lại với phát hiện recon độc lập (Entry #16) rằng hành vi thật của SUT phải khiến case này đỏ.

Đã sửa: Đổi thành `seedAdminToken(page, user.token)` — bơm đúng token của tài khoản `role='user'` vào khoá/origin của app admin, đúng bề mặt mà oracle yêu cầu kiểm tra.

Vì sao AI bỏ sót: Nhầm lẫn giữa hai helper cùng dạng chữ ký (`seedUserToken` / `seedAdminToken`) nhưng khác biệt về *ai đang được seed* so với *origin nào được test*. Đây là lỗi suy luận (chọn sai hàm có sẵn trong cùng file, không phải do thiếu dữ liệu quan sát), và bị bắt được nhờ đối chiếu kết quả pass với một phát hiện recon **độc lập** đã ghi trước đó, không phải nhờ đọc lại code.

2. Nhóm lỗi tin vào chính sản phẩm trước đó của AI

2.1. Hồi quy tự gây ra ở `report:verify` — Entry #5, #9

AI sinh ra: Đổi timestamp trong `utils/stamp.ts` từ dạng UTC (`Z`) sang dạng có offset ICT (`+07:00`) để khớp đúng ngày làm việc thực tế, nhưng không rà lại `scripts/verify-reports.ts`, trong khi công cụ này vẫn dùng regex khớp dạng `Z` cũ.

Vì sao sai: Hai công cụ cùng thao tác trên một định dạng dữ liệu (timestamp trong HTML report) nhưng được sửa không đồng bộ. `report:verify` báo cáo sai (âm tính giả) ngay trên chính dữ liệu đúng.

Đã sửa: Sửa regex trong `report:verify` để chấp nhận cả hai dạng, sau khi phát hiện `report:verify` và `grep` thủ công cho hai kết luận mâu thuẫn nhau ở Entry #9.

Vì sao AI bỏ sót: Giới hạn phạm vi thay đổi: AI sửa đúng file được giao trong Task 2 nhưng không tự mở rộng kiểm tra sang mọi công cụ khác đọc cùng định dạng dữ liệu đó. Bài học áp dụng lại ở Entry #9: khi một công cụ tự kiểm chứng của chính AI báo khác với quan sát trực tiếp, ưu tiên tin quan sát trực tiếp và đi tìm nguyên nhân, không tự động tin công cụ.

2.2. Log "verbatim" nhưng thực chất là tóm tắt — Entry #15

AI sinh ra (ở các phiên trước 08/08): `reports/ai-audit-report.md` và `reports/prompt-log.md` tuyên bố "nguyên văn" ở ngay đầu file, nhưng phần **AI Output** của 4 entry đầu tiên (FR-10) là bản tóm tắt tiếng Việt viết lại sau khi làm xong.

Vì sao sai: Một phiên agentic không có một chuỗi "câu trả lời" duy nhất để dán vào, nên AI chọn viết tóm tắt thay vì đi tìm nguồn dữ liệu thật, vi phạm đúng nguyên tắc mà chính file đó tuyên bố ở dòng đầu.

Đã sửa: Viết `reports/tools/extract-prompt-log.py` để trích trực tiếp từ transcript gốc của Claude Code, và `verify-audit-verbatim.py` để tự động kiểm tra mọi dòng trích dẫn trong `ai-audit-report.md` có thật sự khớp transcript hay không (chạy lại ở mọi entry được thêm trong Session 3, kể cả entry này).

Vì sao AI bỏ sót: Giới hạn thật của kiến trúc agentic (không có một "output" đơn, như Reasoning ở trên đã nêu), kết hợp với việc không có bước kiểm tra chéo tự động cho tới khi sinh viên tự phát hiện mâu thuẫn. Đây là lý do `verify-audit-verbatim.py` được thêm như một bước bắt buộc, không tùy chọn, cho mọi entry từ Session 3 trở đi.

3. Nhóm lỗi oracle nằm trong script chứ không nằm trong dữ liệu

Nhóm này chỉ lộ ra ở lượt rà soát cuối. Nó không làm test đỏ, không làm test xanh sai, và không lộ khi chạy — dấu hiệu duy nhất là trả lời được câu hỏi: *sửa một giá trị trong file JSON thì kết quả test có đổi không?* Chỗ nào trả lời "không" thì chỗ đó dữ liệu chỉ là trang trí.

3.1. Khai báo dữ liệu rồi không đọc tới — Entry #20

AI sinh ra: `loginCaseSchema` có khối `expected` gồm `urlContains` và `tokenStored`. Cả 14 record trong `fr-02-login.cases.json` đều điền đủ hai trường này, và `loadCases()` validate chúng ngay khi nạp.

Vì sao sai: Không dòng nào trong `fr-02-login.spec.ts` đọc tới hai trường đó. Mỗi nhánh `assertion` tự viết lại đúng kỳ vọng ấy bằng hằng số trong code (`toHaveURL(/\/login/)`, `toBeFalsy()`). Nhìn từ bên ngoài, suite có vẻ data-driven đúng chuẩn §6: file dữ liệu tách rời, schema chặt chẽ, không có mảng case inline. Nhưng thứ thực sự quyết định oracle vẫn nằm trong script. Sửa một giá trị trong JSON sẽ không làm test đổi kết quả, mà đó đúng là phép thử để biết một suite có data-driven thật hay không. Hệ quả cụ

thể: hai case không hề kiểm tra URL đích lần trạng thái token (F02-TC-012 và F02-TC-014), dù record của chúng khai báo đầy đủ cả hai.

Đã sửa: Đưa hai assertion đó ra khỏi `switch`, chạy một lần cho mọi case và lấy giá trị kỳ vọng từ chính record. Siết luôn phép so URL: bản cũ khớp chuỗi con `/login`, bản mới neo theo trọn vẹn origin + path, vì so chuỗi con với `/` thì URL nào cũng khớp. Chạy lại đủ 3 trình duyệt cho kết quả không đổi — 10 pass / 4 fail, 4 case đỏ vẫn đúng là F02-TC-002/004/008/012.

Vì sao AI bỏ sót: Không phải lỗi quan sát như nhóm 1 — dữ liệu cần thiết nằm sẵn trong hai file cạnh nhau. Đây là lỗi của việc sinh code theo từng mảnh: stage "Model data" thiết kế schema đầy đủ, stage "Generate" viết spec và tự nghĩ ra assertion từ đầu thay vì tra lại xem schema đã hứa những gì. Không bước nào kiểm tra ngược rằng mọi trường trong schema đều có ít nhất một chỗ đọc tới, và vì test vẫn xanh/đỏ đúng như dự đoán nên không có tín hiệu nào báo động. Lỗi hổng sống qua ba lượt review (Entry #9, #14, #15) trước khi lộ ra ở Entry #20 bằng một câu `grep`.

3.2. Cùng lỗi ấy vẫn còn ở FR-13 — Entry #22

AI sinh ra: `dashboardCaseSchema` có trường `auth` (`admin` / `non-admin` / `anonymous`), cả 12 record đều điền, `loadCases()` đều validate. Nhưng `fr-13-dashboard.spec.ts` rẽ nhánh theo tên `assertion` (`access-denied-anonymous` / `access-denied-non-admin`), không đọc `auth` lần nào.

Vì sao sai: Đúng cùng một khuôn với 3.1, chỉ khác feature. Đổi `auth` của một record từ `admin` sang `anonymous` sẽ không làm test chạy khác đi một chút nào — phiên đăng nhập mà trình duyệt mang theo do script quyết định, không phải do dữ liệu. Đáng chú ý là lỗi này **sống sót qua chính lượt sửa Entry #20**: lượt đó sửa đúng file được chỉ ra (`fr-02-login.spec.ts`) và dừng ở đó, không hỏi tiếp "hai feature còn lại có cùng bệnh không".

Đã sửa: Nhánh FR-12 rẽ theo `testCase.auth`. `assertion` giữ nguyên vai trò "kiểm cái gì", `auth` nhận đúng vai trò "case này là ai" — hai trục độc lập, mỗi trục do dữ liệu quyết định.

Vì sao AI bỏ sót: Giới hạn phạm vi sửa lỗi, giống Entry #5 ở nhóm 2: AI sửa đúng chỗ được chỉ mà không tự tổng quát hoá phát hiện thành một phép kiểm cho toàn suite. Bài học rút ra ở 3.1 đã đúng nhưng chưa được áp dụng đủ rộng, và điều đó chỉ được phát hiện khi rà lại có hệ thống cả ba spec cùng lúc.

3.3. Miền giá trị kỳ vọng viết cứng trong spec — Entry #22

AI sinh ra: `fr-10-order-state.spec.ts` giữ hằng `STATUS_LABEL_DOMAIN` gồm 5 nhãn trạng thái ngay trong spec.

Vì sao sai: Năm nhãn ấy không phải hạ tầng dùng chung — chúng là **kỳ vọng của đúng một case** (F10-TC-012), tức là dữ liệu test viết cứng trong script, đúng thứ §6 cấm. Không nghiêm trọng như 3.1 vì `assertion` vẫn kiểm đúng thứ cần kiểm, nhưng nó khiến case duy nhất ấy không sửa được từ file dữ liệu.

Đã sửa: Chuyển thành `expected.statusDomain` trong record của F10-TC-012; 13 record còn lại khai `null`, và spec dùng `required()` để báo lỗi rõ ràng nếu một case đọc trường này mà record không có.

Vì sao AI bỏ sót: Nhầm lẫn giữa *hằng số của miền nghiệp vụ* và *giá trị kỳ vọng của một case*. Cả hai trông giống nhau trong code; chỉ có câu hỏi "trường này phục vụ mấy case?" mới tách được. Một mảng dùng

đúng một lần thì thuộc về record của case đó.

3.4. Bịa một giá trị hợp lý thay vì thừa nhận nó không tồn tại — Entry #22

AI sinh ra: Trong `resolveIdentity()` của FR-02, nhánh `unregistered` trả về `correctPassword: 'Test1234!'`.

Vì sao sai: Tài khoản `unregistered` theo định nghĩa là chưa từng đăng ký, nên nó **không có** mật khẩu đúng. Hiện tại chưa record nào ghép `unregistered` với `@correct` nên chưa gây hậu quả, nhưng nếu có, test sẽ gửi đi một mật khẩu không ai đăng ký mà vẫn được đọc là "case mật khẩu đúng" — xanh hay đỏ đều vì một lý do dữ liệu không hề nói.

Đã sửa: Trả `null` và ném lỗi kèm tên case nếu một record đòi `@correct` cho một danh tính không có mật khẩu.

Vì sao AI bỏ sót: Xu hướng điển hình một giá trị *trông hợp lý* để kiểu dữ liệu không bị `null`, thay vì để kiểu phản ánh đúng thực tế là giá trị ấy không tồn tại. Đây là biến thể của cùng thói quen ở nhóm 1 (giả định giao diện hợp lý thay vì giao diện thật), lần này áp lên dữ liệu chứ không phải lên DOM.

4. Nhóm lỗi mở rộng phạm vi ngoài yêu cầu (giai đoạn lập kế hoạch, 05/08)

Ghi lại ngắn gọn vì đã xảy ra trước Session 1, nhưng vẫn là gap thật của AI trong toàn bộ vòng đời HW04: viết code khi mới chỉ được yêu cầu đề xuất cấu trúc (Entry #1), và tự ý commit tài liệu nội bộ kèm trailer `Co-Authored-By` dù chưa được yêu cầu (Entry #3) — hành vi thứ hai đặc biệt đáng chú ý vì nó chạm vào lịch sử git, thứ HW04 §12 dùng làm bằng chứng. Cách phòng tránh áp dụng cho toàn bộ phần còn lại của bài: chia kế hoạch thành task có checkpoint rõ ràng, và ghi các ranh giới tuyệt đối (không `Co-Authored-By`, không tự push khi chưa xin phép) vào `CLAUDE.md` để mọi phiên sau đọc lại được thay vì phải nhắc lại mỗi lần.

5. Giới hạn của hạ tầng test tự phát hiện trong Session 3 (không phải lỗi AI sinh code)

`run-matrix.ts` gọi `playwright test` tuần tự cho từng cell, và mỗi lệnh `playwright test` **xoá sạch** `test-results/` khi khởi động. Hệ quả: sau khi chạy đủ 9 cell, `test-results/` chỉ còn giữ ảnh chụp của cell **cuối cùng** (`fr-13-dashboard/webkit`): ảnh gốc của các case đỏ FR-02/FR-10 từ đúng lượt chạy ma trận đã mất trước khi kịp copy làm evidence cho bug report (xem Entry #18). Không phải lỗi trong script test, nhưng là một khoảng trống thật của quy trình: `run-matrix.ts` không lưu attachment nào ngoài report HTML. Cách xử lý trong Session 3 là chạy lại riêng từng case đỏ bằng `--grep` để lấy ảnh sạch, xác nhận lại đúng tên case khớp 100% với lượt chạy ma trận gốc. Việc cải tiến `run-matrix.ts` để tự sao lưu `test-results/` mỗi cell nằm ngoài phạm vi 05/08–08/08 của HW04 và không được thực hiện trong bài này.

6. Tổng kết — điều học được về cộng tác với AI trong kiểm thử

Bốn nhóm lỗi trên không phân tán ngẫu nhiên. Nhóm 1 (giả định giao diện) và nhóm 2 (tin vào sản phẩm trước đó của chính mình) đều là biến thể của cùng một nguyên nhân gốc: AI mạnh ở suy luận trên dữ liệu đã quan sát, và yếu ở việc tự nhận ra khi nó **chưa** quan sát gì cả. Quy trình chống lại điều đó xuyên suốt cả ba feature: recon trên build thật trước khi sinh locator, và đối chiếu hai nguồn độc lập khi chúng mâu thuẫn (Entry #9, #17).

Nhóm 3 thì khác hẳn, và đáng ngại hơn theo một cách riêng: dữ liệu cần thiết nằm sẵn ngay trong repo, không thiếu gì để quan sát, nhưng lỗi vẫn xảy ra vì mỗi stage chỉ nhìn phần việc của mình — stage này khai báo schema, stage kia viết assertion, không stage nào đối chiếu hai bên với nhau. Chính quy trình 7 bước giúp §6 được tuân thủ lại tạo ra đúng loại khe hở này ở ranh giới giữa các bước, và không lượt chạy nào phát hiện được vì test vẫn cho kết quả đúng như dự đoán. Bài học bổ sung: một suite "data-driven" phải được kiểm tra bằng câu hỏi *mọi trường dữ liệu có thật sự được đọc không*, chứ không chỉ bằng việc dữ liệu có nằm ở file riêng hay không.

Điều đáng nói nhất của nhóm 3 là 3 trong 4 trường hợp chỉ lộ ra ở lượt rà soát cuối, **sau khi** trường hợp đầu tiên (3.1) đã được phát hiện và sửa. Lượt sửa đó dừng đúng ở file được chỉ ra. Nói cách khác, ngay cả một phát hiện đúng cũng không tự lan sang phần còn lại của suite nếu không có ai đặt câu hỏi "chỗ khác có cùng bệnh không" — và câu hỏi đó, xuyên suốt cả bài, chưa lần nào do AI tự đặt ra.

Cả bốn nhóm đều dẫn về cùng một kết luận thực hành: không bao giờ tự tin verdict `VALID` chỉ vì output "chạy được". 8/21 entry trong audit report ở mức `INCOMPLETE` chứ không `VALID`, và tỷ lệ đó tự nó là bằng chứng cho thấy review không phải hình thức.